Linux CPU 로드밸런싱 개선 실험 종합 비교

팀 공유 자료 | 2026년 11월 6일 | Linux 5.4.214 기반

현재 가장 유망한 후보는 v9다. 같은 커널 이미지에서 복구 기능을 끄고 켜는 실험을 세 번의 새 부팅에서 반복했고, 168회 측정으로 사전 중앙값 기준을 모두 통과했다. 다만 수정 없는 원본 커널보다 일반 작업까지 안전하게 개선됐다는 최종 결론은 아직 내리지 않는다. [S12]

확인한 항목	현재 결과
복구 지연	ON 중앙값 2.22~6.90 ms 인접 OFF/ON 쌍의 비율 중앙값 기준 85.7~98.9% 감소
일반 작업	조건별 짝 비율 중앙 변화 -2.46~+3.21% 사전 한도 +5% 이내
개별 실행의 편차	복구 24쌍 중 1쌍은 ON이 더 느림 일반 작업 60쌍 중 10쌍은 +5% 초과
최종 판단	같은 이미지의 기능 ON/OFF 검증 통과 원본 커널 대비 전체 채택은 보류

우리가 개선하는 동작

여러 작업을 CPU 하나에 제한했다가 허용 CPU 범위를 넓히면, 기존의 이동 실패로 늘어난 재시도 간격이 남아 분산 회복이 늦을 수 있다. 이 프로젝트는 affinity 확장 사건을 감지해 필요한 재분산을 앞당기는 데 초점을 맞췄다. 새 스케줄러를 만드는 방식이나 VirtualBox 전용 캐시 최적화는 아니다. [S5, S7, S11]

측정 환경

구분	조건
호스트와 VM	Intel Core i5-1334U, 10코어 12스레드 VirtualBox 4 vCPU, 약 4 GiB 메모리
가상화와 배치	최근 진단 로그에서 VT-x 사용 확인 호스트 허용 mask 4095 유지, 물리 코어 고정 미적용
커널과 도구	기준 5.4.214-lb-base, 후보 v4~v9 GCC 9.4, 게스트 Python 3.8.10, schedstats 활성화

최근 우리 VM의 VT-x 확인을 다른 팀원의 Hyper-V 환경에 일반화하지 않는다. 다른 PC의 절대 실행 시간은 개선안 순위를 매기는 기준으로 쓰지 않는다. [S3, S10]

개선 방향이 바뀐 이유

초기에는 cache-hot 작업의 이동 판단을 조절하려 했고, 이후에는 affinity 제한 해제 뒤 재시도 지연을 줄이는 방향으로 옮겼다. v6 이후에는 복구 기능의 효과를 유지하면서 일반 경로의 비용과 부팅 간 변동을 분리하는 일이 핵심이 됐다.

버전	바꾼 부분	후속 판단
v1	CPU_NEWLY_IDLE에서 hotness 시간 기준을 2배로 확대	최근 실행 작업의 이동을 더 오래 억제하는 설계
v2	위 조건에 source CFS runnable 수 > 2 추가	큐에 작업이 많은 경우로 적용 범위 축소
v3	runnable 수 > 1일 때 hotness 기준을 절반으로 축소	새 idle CPU가 작업을 더 빨리 가져오도록 방향 변경
v4	MAX_PINNED_INTERVAL 512 → 32	복구 단축, 계속 고정한 작업의 재시도 증가
v5	affinity 확장 때 기존 NOHZ 경로에 한 번 요청	첫 개선 폭이 새 부팅에서 재현되지 않음
v6	허용 대상 CPU에 irq_work와 로컬 복구 요청	복구 효과 재현, 일반 비용 일부 기준 실패
v7	일반 루프 상수 특수화와 cold handler 분리	복구 유지, 첫 후보 부팅 일반 비용 실패
v8	일반 SCHED_SOFTIRQ 경로 복원, 별도 tasklet 처리	복구 유지, steady와 고정 대조군 비용 실패
v9	기본 OFF인 sysctl 추가, 같은 이미지 안에서 비교	세 부팅의 중앙값 기준 통과, 최종 채택 보류

v1~v3은 task_hot 패치의 설계 이력으로 구분한다. 이 표에서 v4 이후의 정식 검증과 같은 수준의 성능 순위를 부여하지 않는다. 벤치마크 도구의 v2와 커널 후보 v2는 서로 다른 버전 이름이다. [S2]

초기 기준값부터 변동이 컸다

커널을 바꾸지 않은 1024 KiB 재측정에서도 중앙값이 7.852초에서 6.149초로 약 21.7% 줄었다. 새 묶음 내부 변동은 2.47%로 작았지만, 시점이 다른 묶음끼리는 큰 차이가 남았다. 이동 횟수도 2,195회에서 2,479회로 늘면서 더 빨라져, 이동 감소를 곧 성능 향상으로 해석할 근거가 없었다. [S1]

실행 큐를 주기적으로 읽는 계측도 영향을 줬다. bursty 1024 KiB에서 전체 큐를 읽은 경우 관찰/대조 중앙값 비율은 +13.69%였고, 작업자 상태만 읽으면 +3.66%였다. 이 결과는 고정된 계측 비용의 추정치가 아니다. 이후 일반 성능 측정과 별도의 복구 probe를 구분했다. [S4]

v4와 v5의 효과와 재현 문제

v4는 재시도 간격을 전역으로 줄였다

MAX_PINNED_INTERVAL을 512에서 32로 줄이면 제한이 풀린 뒤 다음 재분산 기회가 빨라질 수 있다. 이 값은 실제 대기 시간이 언제나 32 ms 이하라는 뜻이 아니다. 도메인 간격과 busy factor 등도 다음 실행 시점에 관여한다. [S5]

재검증 조건	앞 기준 A2	v4 B2	뒤 기준 A3
200 ms 고정 뒤 복구	41.73 ms	23.73 ms	52.51 ms
800 ms 고정 뒤 복구	82.36 ms	35.20 ms	75.93 ms
bursty 1024 KiB	7.343 s	8.008 s	7.115 s
계속 고정 idle balance 시도율	13.6 /s	90.9 /s	12.1 /s

정식 비교는 다섯 부팅에서 139회였다. 복구 중앙값은 재검증에서 43.1~57.3% 줄었지만 bursty 실행은 앞뒤 기준보다 9.1~12.5% 느렸고, 계속 고정한 상태의 재시도율은 약 6.7~7.5배로 늘었다. 첫 비교의 16.1% 실행 시간 단축은 재현되지 않아 개선 근거로 채택하지 않았다. [S5]

v5는 확장 사건이 있을 때만 요청했다

v5는 원래의 backoff 상한과 task_hot 판단을 유지하고, CPU 허용 마스크가 실제로 넓어질 때 기존 NOHZ idle load balancer에 한 번 요청했다. 계속 제한된 작업에 불필요한 재시도를 늘리는 문제를 피하려는 접근이었다. [S6]

복구 조건	첫 후보 B1	추가 후보 B2	추가 기준 A3
200 ms 고정 뒤 복구	14.04 ms	52.62 ms	56.62 ms
800 ms 고정 뒤 복구	18.29 ms	55.05 ms	55.04 ms

누적 205회에서 첫 후보의 70~82% 감소는 반복되지 않았다. 새 B2/A3 비교에서는 약 7.1% 단축과 0.02% 악화로 효과가 작아졌다. 요청과 처리 카운터가 두 후보에서 같아도 실제 분산 완료가 같지는 않았다. v5는 채택을 보류했다. [S6]

v5 초기 비교는 system service/runuser, 추가 비교는 사용자 터미널이었고 실제 시작 uptime도 달랐다. 새 B2/A3를 우선 비교하며, 초기와 추가 결과를 완전히 같은 실행 맥락으로 묶지 않는다. 이는 v6 이후 실행 맥락과 시작 시점을 맞춘 이유다.

v6부터 v8까지 복구 효과는 유지됐다

v6는 허용 대상 CPU에 직접 알리고, 대상에서 기존 load balancer를 한 번 앞당겼다. v7은 일반 경로에서 추가 조건을 줄였고 v8은 복구 처리를 별도 tasklet으로 옮겼다. 세 후보 모두 복구 효과는 두 후보 부팅에서 유지됐으나 전체 비용 기준은 통과하지 못했다. [S7~S9]

후보	측정 규모	후보 복구 중앙값	인접 기준 대비 감소
v6	5부팅 205회	1.76~2.96 ms	93.0~98.7%
v7	5부팅 325회	1.55~3.24 ms	95.1~98.5%
v8	5부팅 325회	2.25~3.60 ms	94.2~98.1%

각 후보 두 부팅과 세 기준 부팅을 A1 → B1 → A2 → B2 → A3로 비교했다. 공유되는 A2를 중복 부팅으로 세지 않는다. 위 감소율은 각 후보와 인접 기준의 중앙값 비율이다. v9의 짝별 비율 중앙값과 계산 방식이 다르다.

복구 함수가 아니라 일반 작업에서 비용 기준을 넘었다

후보	사전 기준을 넘은 비교	판정
v6	B2/A3 bursty 256 KiB +5.22% B2/A3 steady +5.64%	일반 작업 2개 비교 실패
v7	B1 steady +6.86~8.30% B1 CPU0 고정 +8.48~12.64% B1/A2 bursty 1024 KiB +5.94%	B2는 통과했어도 전체 보류
v8	steady 일부 +7.08%, +11.86% CPU0 고정 비교 +12.29~23.08%	bursty 통과, 전체 보류

코드 구조를 확인한 의미

v7은 공통 루프를 상수 인자로 특수화하고 복구 handler를 cold 함수로 분리했다. 컴파일 결과 일반 루프의 affinity 계측 참조가 없어지고 run_rebalance_domains의 명령 줄 수가 178에서 39로 줄었지만, 이 숫자가 실제 실행 시간 감소율을 뜻하지는 않는다. [S8]

v8은 일반 rebalance_domains, run_rebalance_domains, trigger_load_balance의 소스를 기준으로 되돌렸다. 두 함수의 오브젝트 바이트는 같았고, 나머지 함수는 같은 호출 대상의 상대 주소와 UBSAN 메타데이터 위치가 달랐다. 호출 함수를 포함한 네 함수의 명령 흐름과 호출 대상 검사는 통과했다. 구조가 같아도 전체 커널 실행 비용이 같다는 증명은 아니다. [S9]

일반 단계에서 새 이벤트 카운터가 0이었다는 사실은 새 복구 호출이 없었다는 뜻이다. 코드 배치, 다른 경로, 앞선 복구의 상태 영향, 호스트 실행 조건까지 영향이 없었다는 뜻은 아니다.

일반 작업이 느려진 원인을 좁힌 진단

v8 전체 비교에서 관측한 실행 시간

조건	기준 A1	v8 B1	기준 A2	v8 B2	기준 A3
bursty 64 KiB	3.909	3.924	3.949	3.927	3.858
bursty 256 KiB	4.774	4.773	4.832	4.757	4.705
bursty 1024 KiB	8.131	8.249	8.583	8.188	7.958
steady 256 KiB	3.670	3.930	3.966	4.043	3.614
CPU0 계속 고정	10.958	13.487	11.273	12.659	10.838

단위 초, 각 부팅과 조건의 중앙값이다. 이 325회 결과의 비용 기준 실패는 뒤의 별도 진단으로 대체하지 않았다. [S9]

같은 원본 커널도 부팅 사이에 크게 달라졌다

별도 진단은 기준 → v8 → 기준으로 180회 실행했다. 작업자 수와 버퍼를 바꾸고 게스트 CPU0~3에 각각 고정했으며, 게스트 wall·worker CPU·user/system 시간과 Windows VM 프로세스 CPU 시간을 함께 기록했다. 계산 출력 189개와 독립 호스트 marker 360개를 검증했다. [S10]

CPU0 진단 조건	기준 A1	v8	기준 A2
1 worker 4 KiB	0.6539 s	0.5347 s	0.5303 s
1 worker 256 KiB	1.3756 s	0.9732 s	0.9984 s
8 workers 256 KiB	15.0190 s	10.3270 s	10.4571 s

12개 진단 조건에서 v8은 뒤 기준 A2와 비교하면 -2.52~+3.16%였지만 앞 기준 A1과는 훨씬 큰 차이가 났다. 같은 원본의 CPU0 고정 시간이 15.02초에서 10.46초로 바뀐 것은 부팅과 실행 시점의 변동을 먼저 고려해야 한다는 근거다. [S10]

아직 분리하지 못한 영향

호스트 CPU 시간도 함께 바뀌었으므로 게스트 시계 오차 하나로 설명할 수 없다. 반대로 VM 전체 프로세스 CPU 시간은 GUI와 장치 처리까지 포함하고 cycle 수가 아니어서, 실제 커널 명령 비용 증가의 증명도 아니다. 게스트 taskset은 호스트 물리 코어를 고정하지 않으며 P/E 코어 배치, 클럭, 캐시 중 어느 요인이 원인인지 확인하지 못했다.

진단에는 앞선 recovery 단계가 없었고 전체 suite는 recovery 후 일반 작업을 측정했다. 앞선 복구가 남긴 상태 영향도 후보 가설이다. 이 때문에 v9에서는 같은 이미지와 부팅 안에서 상태 순서를 섞어 비교하도록 바꿨다. [S10, S11]

v9 구현과 비교 방법

허용 CPU 확장 사건에만 반응한다

v9는 v8의 복구 구조를 유지하고 affinity.c에 기본값 0인 kernel.sched_affinity_rebalance 제어를 추가했다. root만 0 또는 1을 쓸 수 있고, 제어값은 확장 알림과 tasklet 경로에서 읽는다. 일반 balancing 함수에는 상태 검사를 추가하지 않았다. [S11]

- 실행 가능한 CFS 작업의 허용 마스크가 strict superset으로 넓어지고 source 큐에 둘 이상의 CFS 작업이 있을 때 감지한다.
- source를 제외한 허용 active CPU 중 도메인 조건을 만족하는 대상에 irq_work를 보낸다. hardirq callback은 tasklet만 예약한다.
- tasklet에서 해당 도메인의 balance_interval을 min_interval로 되돌리고 기존 load balancer를 호출한다. 이동 허용 검사는 그대로 사용한다.
- MAX_PINNED_INTERVAL과 task_hot 정책은 기준대로다. 별도 빠른 주기 타이머는 추가하지 않는다.

OFF 전에 예약된 tasklet도 처리 시 상태를 확인한다. 이미 실행 중인 callback이 즉시 취소된다는 뜻은 아니므로 상태 전환 뒤 1.5초 대기와 카운터 대조를 포함했다.

같은 이미지 안의 인접 쌍으로 비교한다

C 프로그램은 전체 2,086,169,180 steps를 작업자 8명에게 나눠 계산한다. 각자 독립 버퍼를 쓰고, steady는 계속 계산하며 bursty는 계산 묶음 사이에 1~3 ms 수면을 요청한다. 버퍼 크기는 작업자 1명당 값으로 8명 × 1024 KiB이면 합계 8 MiB다. affinity 제어는 C 본체와 별도의 probe 및 실행기가 담당한다. [S3, S11]

항목	고정한 조건
이미지와 작업	5.4.214-lb-v9 동일 이미지 원본 C·바이너리·복구 probe 유지, 일반 단계 8 workers
7개 조건	복구 hold 200/800 ms, bursty 64/256/1024 KiB steady 256 KiB, CPU0 계속 고정 256 KiB
반복과 순서	조건별 4쌍, OFF-first 2쌍과 ON-first 2쌍 7조건 × 4쌍 × 2상태 = 부팅당 56회
재현	동일 child protocol로 새 부팅 3회, 총 168회 각 부팅을 따로 평가, 시작 uptime 60.08~60.20 s
맥락과 계측	같은 부팅의 OFF/ON은 같은 service와 사용자 경로 복구 hold ±64 ms jitter를 짝 안에서 동일하게 적용, snapshots OFF

사전에 정한 통과 기준

일반 작업의 짝별 시간 비율 중앙값은 ON/OFF ≤ 1.05, 복구는 ≤ 0.80으로 정했다. affinity 설정 batch를 더한 ON 포함 응답 중앙값은 10 ms 이하, 계속 고정한 상태의 idle balance 시도율 비율은 2 이하로 정했다. OFF의 새 이벤트는 모두 0, ON 일반 작업도 0, ON 복구에서는 네 카운터가 모두 양수여야 한다. [S11, S12]

v9 세 부팅의 수치와 개별 실행 편차

아래 복구 시간은 worker가 관측한 CPU 배치에서 50 ms 지속 커버가 시작한 지연이다. 정확한 migration 완료 시각이나 원자적인 실제 CPU 점유 상태를 측정한 값은 아니다. 포함 응답은 affinity 설정 batch 시간을 더한 보수적 지표다. [S12]

부팅	hold ms	OFF ms	ON ms	짝별 변화	ON 포함 ms
파일럿	200	174.552	2.221	-98.90%	2.494
파일럿	800	78.535	2.581	-95.98%	3.056
재현 1	200	41.891	3.622	-85.89%	3.831
재현 1	800	165.617	2.797	-93.79%	2.968
재현 2	200	127.173	5.040	-96.98%	5.193
재현 2	800	30.566	6.902	-85.71%	7.120

OFF와 ON은 각각 4회의 중앙값이다. 짝별 변화는 각 인접 쌍에서 ON/OFF - 1을 계산한 뒤 중앙값을 취했다. 상태별 중앙값의 비율과 같지 않을 수 있다.

일반 작업의 짝별 실행 시간 변화

조건	파일럿	재현 1	재현 2
bursty 64 KiB	+0.31%	+0.81%	-0.16%
bursty 256 KiB	-0.67%	+0.61%	+0.34%
bursty 1024 KiB	-2.08%	-2.46%	-1.25%
steady 256 KiB	+3.21%	-0.66%	+1.87%
CPU0 고정 256 KiB	-1.41%	+3.07%	+2.06%

세 부팅 모두 사전 중앙값 기준을 통과했다. 일반 작업의 조건별 중앙 변화는 -2.46~+3.21%였다. 이 범위는 개별 실행의 최솟값과 최댓값이 아니며, 일반 처리량이 일관되게 향상됐다는 뜻도 아니다.

중앙값이 통과해도 느린 실행은 남는다

복구 24쌍 중 1쌍은 ON이 더 느렸다. 재현 2의 800 ms 조건 두 번째 쌍은 OFF 28.27 ms, ON 47.05 ms로 66.41% 악화였다. ON 단일 복구 시간 최댓값은 47.05 ms, 포함 응답 최댓값은 47.13 ms였다.

일반 작업 60쌍 중 10쌍은 +5%를 넘었다. 가장 큰 차이는 파일럿 CPU0 고정 세 번째 쌍으로 13.55초 → 18.39초, +35.71%였다. 이 실행들을 제외하지 않았다. 사전 기준은 쌍별 최대가 아니라 중앙값이므로 통과 판정과 모순되지는 않지만, 매 실행의 성능 보장이나 tail 개선의 근거는 되지 않는다. [S12]

현재 판단과 다음 비교

무엇까지 말할 수 있는가

affinity 확장 후 복구 지연을 줄이는 기능은 v6~v8의 두 후보 부팅과 v9의 세 부팅에서 반복 관측됐다. v9에서는 같은 이미지의 ON/OFF 중앙값 비교에서 일반 작업 비용도 사전 한도 안이었다. 따라서 현재는 복구 전용 개선 후보와 그 비교 방법을 확보한 상태다.

OFF 상태에도 실험 코드와 코드 배치가 들어 있다. v9 OFF/ON은 기능을 켜는 증분 효과를 비교할 수 있지만, 수정 없는 원본 커널에 패치 전체를 추가하는 비용을 모두 제거해 측정한 것은 아니다. 기존 v8의 비용 기준 실패는 유지하며, v9 기본 채택은 보류한다. [S9, S12]

다음 단계는 원본과 OFF와 ON의 세 상태 비교다

순서	추가할 비교	확인할 질문
1	수정 없는 원본 A v9 OFF B, v9 ON C	OFF의 코드 배치 등 공통 비용과 기능 ON의 증분 효과가 다른가
2	같은 PC와 VM에서 새 부팅을 교차 모든 상태를 같은 service로 실행	기준 부팅의 시간 변동에도 일반 작업 비용 한도를 지키는가
3	recovery 선행 유무를 별도 조건으로 고정	앞선 복구의 상태가 뒤 일반 작업 시간에 영향을 주는가
4	충분한 반복 수와 tail 기준 사전 정의	느린 실행이 줄고 특정 실행의 큰 악화가 반복되는가
5	별도 호스트와 가상화 backend에서 재현	각 환경의 원본 대비 효과가 같은 방향으로 나오는가

위 내용은 다음 실험 제안이다. 이 문서 작성 중 커널이나 벤치마크를 수정하거나 새 측정을 시작하지 않았다. 반복 수와 통과 기준은 결과를 보기 전에 확정한다.

팀에서 나눌 일

- A·D 담당: v9 코드와 원본 대비 패치를 검토하고, 원본·OFF·ON 이미지와 설정을 준비한다. 잠금, CPU hotplug, 복잡한 cgroup과 유지보수용 중복 루프도 검토한다.
- B·C 담당: 기존 C와 workload를 유지하고, 세 상태의 교차 실행 계획과 recovery 선행 여부를 고정한다. checksum·이미지·카운터·호스트 marker를 검증하고 중앙값과 개별 시행을 함께 비교한다.

팀원에게 전달할 결론

현재 v9는 affinity 해제 뒤 복구를 빠르게 하는 후보로 유망하다. 같은 이미지의 OFF/ON을 세 부팅에서 반복해 중앙값 기준을 통과했지만, 일부 개별 실행은 느렸고 원본 커널 대비 전체 비용 검증은 남아 있다. 캐시 최적화나 전체 처리량 향상을 입증한 것으로 발표하지 않는다.

검증 기록과 근거 자료

코드와 계산 결과를 유지했다

원본 C의 계산 루프와 바이너리는 수정하지 않았다. v9 세 부팅의 정식 측정 168회에 더해 기준 출력과 워밍업을 포함한 계산 출력 189개, 독립 호스트 marker 336개를 검증했다. 서로 다른 /proc/stat btime 세 개를 확인했고 모듈 이름 집합도 같았다. 커널 오류 패턴 검사에서 UBSAN·KASAN·BUG·Oops·lockup·RCU stall 징후는 없었다. 이는 모든 결함이 없다는 보증은 아니다. [S12]

최초 export 검증에서는 CPU 합계 sum_cpu_s의 부동소수점 끝자리 차이 11건이 발견됐다. 최대 차이는 약 7.1×10^-15초였고 checksum은 일치했다. 원본 validator와 원자료를 유지한 채 별도 auditor가 게스트의 순차 합산을 재현해 전체 수치를 정확히 대조했다. 허용 오차와 성능 기준은 추가하지 않았다. [S13]

실험 종료 뒤 기준 커널 5.4.214-lb-base 복귀, 재현 서비스 disabled/inactive, 완료 상태 complete를 직접 확인했다. 임시 NAT와 호스트 수집 프로세스도 정리했다. [S12]

근거 자료 위치

아래 경로는 Linux 프로젝트 폴더를 기준으로 한다. 버전별 README의 이전 상태 설명보다 해당 RESULTS와 최신 comparison 및 검증 파일을 우선한다.

번호	확인할 자료
S1~S4	docs/Assignment/CPU_load_balancing_team_plan.md loadbalance-v2/TEAM_GUIDE.md, OBSERVE_BALANCE.md output/team_project/lb-candidate/newidle-locality-v1~v3.patch
S5~S6	loadbalance-v2/BACKOFF_V4.md output/team_project/lb-v5/RECHECK_RESULTS.md
S7~S9	output/team_project/lb-v6/, lb-v7/, lb-v8/ 각 README.md, RESULTS.md, comparison-vN.json
S10	output/team_project/lb-cpu-diagnosis/RESULTS.md host-environment.json 및 results/
S11	output/team_project/lb-v9/README.md, protocol.json affinity-control-v9.patch loadbalance-v2/scripts/run_paired_v9.py, prepare_v9_pilot.py
S12	output/team_project/lb-v9-repro/comparison.json VALIDATION.json, boot-identity-audit.json, FINAL_VM_STATE.json
S13	output/team_project/lb-v9/POSTPROCESSING_CORRECTION.json

v9 코드와 원자료 묶음: output/team_project/loadbalance-affinity-v9-repro-20261106.zip. 이 문서의 수치 및 참조 파일 SHA256은 함께 생성한 비교 문서 근거 JSON에 보존했다.